



Solar ASIC
AUTOMATISATION

Guia Instalacion Completa

Automatiza tus mineros ASIC con energia solar.
Cero consumo de red de dia | Minería solar | A prueba de bear market.

Solar

Cripto

Home Assistant

Automatizacion V1.0.37



VERSION

1.0.37

AUTOR

Halo44

LICENCIA

Gratis & Libre

HARDWARE

Refoss + HA

Indice

- 01** **El Concepto**
Por que Solar ASIC? Bear market + solar

- 02** **Arquitectura**
Vision general del sistema completo

- 03** **Hardware & Cableado**
Refoss EM06P, enchufes, colocacion CT

- 04** **Home Assistant**
Integracion y configuracion base

- 05** **configuration.yaml**
Sensores, plantillas, ASIC prioritarios, ingresos

- 06** **Automatizacion V1.0.37**
Multi-prioridad, anti yo-yo, SHED continuo

- 07** **Informe diario**
Notificacion 20h fiable en HA (history_stats)

- 08** **Dashboard HTML**
Interfaz en vivo, graficos kWh, i18n

- 09** **Pools de mineria**
7 pools: F2Pool, K1Pool, 2Miners, ViaBTC...

- 10** **Acceso remoto**
Tailscale VPN - acceso 4G / WiFi

- 11** **Resolucion de problemas**
FAQ y solucion de problemas

01 El Concepto

Por que Solar ASIC?

En un bear market cripto, minar con electricidad de red a 0,20 EUR/kWh suele ser a perdida. La solucion: usar solo la energia solar excedente de tu instalacion. Mientras haya excedente solar, los ASIC funcionan gratis. En cuanto el excedente baja, se apagan automaticamente para no consumir nunca de la red.

El ciclo diario automatico

Momento	Comportamiento
Manana	Amanecer -> excedente creciente. Los ASIC pequenos arrancan uno a uno (cascada 5s).
Dia	Excedente maximo. La automatizacion cambia a los ASIC grandes prioritarios (S19, S21...). Mineria 100% solar, cero red.
Tarde	El excedente baja. Los ASIC se apagan progresivamente (orden inverso, LIFO).
Noche	Todos los ASIC apagados. Sin consumo de red.

Diferencia clave vs v1: la automatizacion V1.0.37 gestiona una **lista de ASIC prioritarios**. Por la manana, los ASIC pequenos llenan el excedente; en cuanto hay suficiente sol para un minero grande, los pequenos se cortan mientras arranca, y luego vuelven en el excedente restante. Cascada automatica para 1, 2, 3 ASIC grandes o mas.

Tres pilares

- **Solar primero:** la produccion FV se mide en tiempo real con 6 pinzas CT.
- **Decision inteligente:** Home Assistant calcula el excedente y controla cada enchufe.
- **Anti yo-yo:** retardos y cooldowns evitan encendidos/apagados por nubes pasajeras.

02

Arquitectura

Vision general del sistema

Cinco eslabones conectan tus paneles con tus ASIC. Cada uno tiene un papel preciso en la cadena de medicion y decision.

Eslabon	Papel
1. Paneles FV	Producen la energia (ej: 12 kWp)
2. Refoss EM06P	6 pinzas CT miden red (CT 1-2-3) y granja ASIC (CT 4-5-6)
3. Home Assistant	Cerebro: calcula el excedente, decide que ASIC funcionan (Raspberry Pi)
4. Enchufes	Ejecutan: WiFi / Zigbee / Matter (Tuya, Shelly...)
5. Mineros ASIC	Minan: IceRiver, Goldshell, Antminer...

Flujo de datos



Ventaja	Detalle
Automatico 24/7	La automatizacion funciona en continuo, ciclo cada 5 segundos
Anti yo-yo	Cooldown 30s + arranque prioritario 10 min
Acceso remoto	Tailscale VPN para consultar en 4G
Coste de red	0 EUR para minar durante el dia

03

Hardware & Cableado

Esquema de cableado - pinzas CT Refoss

El Refoss EM06P tiene 6 canales de medicion (6 pinzas de corriente). Su colocacion es crucial para que los calculos de excedente sean correctos.

Pinzas	Colocacion	Medicion
CT 1, 2, 3	Cables de red, entre el contador y el inversor	Importacion/exportacion de red (signo: + importacion, - exportacion)
CT 4, 5, 6	Cables de salida del inversor (lado AC)	Consumo de la granja ASIC (siempre positivo)

IMPORTANTE: el cable de tension del Refoss (L1 / L2 / L3 / N) debe conectarse al mismo circuito que los CT 1-2-3, de lo contrario las mediciones de potencia seran erroneas.

Lista de hardware

Elemento	Modelo recomendado	Uso
Medidor CT 6 canales	Refoss EM06P (o 2x Shelly 3EM)	Medir solar + red
Enchufes	Tuya / Shelly / Matter, 1 por ASIC	Encendido/apagado controlado
Mini-PC / Pi	Raspberry Pi 3B+ minimo o PC	Aloja Home Assistant
Mineros ASIC	IceRiver, Goldshell, Antminer...	A eleccion segun presupuesto

Consejo enchufes: verifica el amperaje maximo de cada enchufe. Un Antminer S19 (~3400W) necesita un enchufe 16A dedicado. Los pequenos IceRiver (~100W) van en enchufes 10A estandar.

04 Home Assistant

Configuracion base

Antes de programar los sensores, integra el hardware y prepara el entorno HA.

Pasos preliminares

- 1. Integracion Refoss:** Ajustes -> Dispositivos y servicios -> Anadir -> Refoss. Aparecen los 6 sensores sensor.em06_channel_X_power.
- 2. Integracion de enchufes:** anade tus enchufes (Tuya, Shelly...). Anota sus entity_id (Herramientas dev -> Estados -> filtrar 'switch.').
- 3. Token de larga duracion:** Perfil -> Seguridad -> Crear token. Copialo para el dashboard.
- 4. Carpeta www:** crea /config/www/ si no existe (para alojar el dashboard).

CORS: para que el dashboard HTML acceda a la API de HA, anade el bloque http: con cors_allowed_origins en configuration.yaml (ver seccion 05).

Recorder (historial)

El dashboard muestra graficos dia/semana/mes. Para ello, HA debe conservar el historial de los sensores durante al menos 31 dias:

```
recorder:
  purge_keep_days: 31
  commit_interval: 60
  include:
    entities:
      - sensor.production_solaire
      - sensor.consommmation_ferme
      - sensor.consommmation_reseau
      - sensor.injection_reseau
      - sensor.asic_allumes_count
```

05 configuration.yaml

Sensores calculados (plantillas Jinja2)

Estos sensores transforman las mediciones brutas de los CT en valores utiles: produccion solar, excedente, importacion/exportacion de red, recuento ASIC.

Lo que debes adaptar

- **sensor.em06_channel_X_power** -> los nombres reales de tus sensores Refoss
- **switch.asic_small_1 ...** -> los entity_id de tus enchufes
- **100, 360, 3400** -> el consumo real (W) de cada ASIC

```
template:
- sensor:
  - name: "reseau_total"          # CT 1-2-3 (+ import / - export)
    unit_of_measurement: "W"
    state: >
      {% set p1 = states('sensor.em06_channel_1_power')|float(0) %}
      {% set p2 = states('sensor.em06_channel_2_power')|float(0) %}
      {% set p3 = states('sensor.em06_channel_3_power')|float(0) %}
      {{ (p1 + p2 + p3) | round(1) }}

  - name: "injection_reseau"      # surplus to grid
    state: >
      {% set p = states('sensor.reseau_total')|float(0) %}
      {{ (p * -1) if p < 0 else 0 }}

  - name: "consommation_ferme"    # CT 4-5-6 (ASIC load)
    state: >
      {% set p4 = states('sensor.em06_channel_4_power')|float(0) %}
      {% set p5 = states('sensor.em06_channel_5_power')|float(0) %}
      {% set p6 = states('sensor.em06_channel_6_power')|float(0) %}
      {{ [p4 + p5 + p6, 0] | max | round(1) }}

  - name: "production_solaire"    # farm + export
    state: >
      {% set f = states('sensor.consommation_ferme')|float(0) %}
      {% set i = states('sensor.injection_reseau')|float(0) %}
      {{ (f + i) | round(1) }}
```

ASIC prioritarios (fuente de verdad)

El corazon del sistema V1.0.37: una sola lista define que ASIC son prioritarios (los mineros grandes). La automatizacion lee esta lista y gestiona todo automaticamente. Para anadir un ASIC (ej: un nuevo S21), basta con anadir una linea aqui - sin modificar la automatizacion.

```
- name: "asic_prio_config"
  state: "ok"
  attributes:
    asics: >-
      {{ [
        {'entity': 'switch.asic_big_1', 'power': 3400, 'name': 'S19'}
      ] }}
  # Multiple priorities:
  # {'entity': 'switch.s19', 'power': 3450, 'name': 'S19'},
  # {'entity': 'switch.s21_1', 'power': 3500, 'name': 'S21 #1'},
  # {'entity': 'switch.s21_2', 'power': 3500, 'name': 'S21 #2'}
```

Cinco sensores derivados (copiar tal cual) calculan automaticamente: la potencia necesaria para el proximo arranque, el proximo ASIC a encender, el ultimo encendido y el contador de ASIC prioritarios activos.

Sensor	Papel
<code>asic_prio_power_needed</code>	Potencia total requerida para arrancar el proximo prioritario
<code>asic_prio_next_entity</code>	entity_id del proximo ASIC a encender (FIFO)
<code>asic_prio_last_entity</code>	entity_id del ultimo encendido (primero a cortar, LIFO)
<code>asic_prio_count_on</code>	Numero de ASIC prioritarios actualmente ON
<code>binary_sensor.asic_prio_add_ready</code>	ON cuando la produccion solar cubre el proximo prioritario

Ingresos de mineria en euros

Dos sensores plantilla multiplican la produccion de cada pool (en cripto) por el precio en euros obtenido via CoinGecko, para mostrar los ingresos reales en EUR.

```
- platform: rest          # crypto prices, 1 call / 10 min
  name: "crypto_prices_eur"
  resource: "https://api.coingecko.com/api/v3/simple/price
    ?ids=bitcoin,kaspa,alephium&vs_currencies=eur"
  scan_interval: 600
  json_attributes: [bitcoin, kaspa, alephium]
```

06

Automatizacion V1.0.37

Logica multi-prioridad

La automatizacion V37 funciona en modo single (una instancia a la vez) con 5 disparadores. No contiene ningun nombre de ASIC fijo: todo viene de sensor.asic_prio_config.

Disparador	Condicion	Accion
cycle_normal	Cada 5s	Regulacion continua: anade/quita ASIC pequenos
maybe_prio_add	Prod >= necesidad, 1 min	Arranca el proximo ASIC prioritario
sustained_import	Red > 250W, 5 min	Reduce los ASIC pequenos
production_drop	Prod < 2800W, 5 min	Corta el ultimo prioritario (tarde)
endofday_cleanup	Red > 50W, 10 min	Corta todos los ASIC pequenos restantes (noche)

Secuencia de arranque de un prioritario

Cuando la produccion solar cubre la potencia del proximo ASIC grande durante 1 minuto:

- Todos los ASIC pequenos se cortan (liberar el bus 230V)
- Despues de 25s, el ASIC grande arranca
- Durante 10 min, anadir ASIC pequenos esta **bloqueado** (el grande sube ~5 min)
- Una vez estabilizado, los pequenos vuelven en el excedente restante

Por que el bloqueo de 10 min? Un ASIC grande (S19) no consume sus 3400W al instante: tarda ~5 min en subir. Sin el bloqueo, el ciclo ADD veria un falso excedente, encenderia ASIC pequenos, y luego deberia cortarlos cuando el grande alcanza plena potencia -> importacion de red y yo-yo. La variable prio_boot_ok lo evita.

SHED continuo - la correccion clave

El disparador sustained_import solo se activa una vez. Para reaccionar a una importacion persistente, cycle_normal integra una rama SHED: si red > 50W y quedan ASIC pequenos, corta uno cada 5s (con cooldown 30s). Esto lleva el consumo por debajo de la produccion progresivamente, sin yo-yo.

```
# In cycle_normal:
- choose:
  - conditions:
    - "{{ grid_total > 50 and nb_small_on > 0
      and switch_cooldown_ok }}"
  sequence:
    - action: switch.turn_off
      target: {entity_id: "{{ small_on[-1] }}"}
```

07 Informe diario

Informe fiable en el lado de Home Assistant

Cada día a las 20h, una automatización HA envía un informe completo por Telegram, ntfy y la app HA Companion. A diferencia de una versión navegador, funciona 24/7: las duraciones de minería son por tanto siempre exactas, incluso si el dashboard está cerrado.

Sensores history_stats

Home Assistant calcula en continuo el tiempo ON de cada ASIC (reset a medianoche). Estos sensores alimentan el informe con duraciones fiables.

```
sensor:
  - platform: history_stats
    name: "asic_big_1_heures_jour"
    entity_id: switch.asic_big_1
    state: "on"
    type: time
    start: "{{ today_at('00:00') }}"
    end: "{{ now() }}"
```

Contenido del informe

Bloque	Detalle
Energía	Solar producido, carga ASIC, importación/exportación de red (kWh)
Duraciones	Minería total + horas por ASIC, agrupadas por modelo (history_stats)
Ingresos	Ingresos de minería del día anterior (D-1) en euros
Neto	Ingresos - coste de red (exportación no contada sin contrato de recompra)

Neto sin exportación: sin contrato de recompra EDF, la energía exportada no se remunera. Neto = ingresos de minería - coste de importación de red. La exportación se muestra a título informativo.

Canales de notificación

- **Telegram:** vía un bot (token + chat_id), mensaje instantáneo
- **ntfy:** push de código abierto, topic personalizado (app ntfy.sh)
- **HA Companion:** notificación nativa en la app móvil de Home Assistant

08 Dashboard HTML

Interfaz visual en tiempo real

Un unico archivo HTML (HTML + CSS + JS) para colocar en /config/www/. Lee la API de HA y muestra produccion, consumo, flujo de energia animado, ingresos de mineria e historial.

#	Paso	Detalle
1	Copiar el archivo	/config/www/dashboard_mining.html
2	Copiar secrets.example.js	-> secrets.js (tus tokens e IDs)
3	Crear un token API	Perfil HA -> Seguridad -> Tokens de larga duracion
4	Abrir el dashboard	http://IP_HA:8123/local/dashboard_mining.html

Variables a personalizar (secrets.js)

Variable	Por defecto	Reemplazar por
HA_URL_LOCAL	http://192.168.1.XX:8123	IP local de tu HA
HA_URL_VPN	http://100.X.X.X:8123	IP Tailscale (opcional)
HA_TOKEN	eyJhbGci...	Tu token API de HA
MINING_COINS	[...]	Lista de tus criptos minadas
F2POOL_TOKEN	tu_token	Clave API del pool

Funcionalidades del dashboard

- **Flujo de energia animado:** solar -> granja -> ASIC, con importacion/exportacion de red
- **Grafico dia:** curvas de potencia (W) sobre 24h
- **Grafico semana/mes:** barras kWh diarias (estilo Solarman)
- **Ingresos de mineria:** por cripto, en tiempo real y en euros
- **Informe ayer:** ingresos - coste de red del dia anterior
- **Multi-idioma:** frances, ingles, aleman, espanol
- **Proximo arranque:** cuenta atras y umbral del proximo ASIC

Graficos de barras: para las vistas de semana y mes, el dashboard integra la potencia (W) en energia (kWh) por dia mediante el metodo trapezoidal, y luego muestra 4 barras por dia (solar, granja, importacion, exportacion) - igual que la app Solarman.

09 Pools de minería

7 pools soportados

Solar ASIC puede mostrar los ingresos de cualquier pool. La arquitectura es universal: Home Assistant obtiene los datos del lado del servidor (sin problema CORS), el dashboard lee los sensores de HA. Todos los sensores deben llamarse `sensor.f2pool_SIMBOLO_day` y `sensor.f2pool_SIMBOLO_unpaid` (el prefijo `f2pool_` es convencional, sea cual sea el pool real).

Pool	Criptos	Auth	Tipo	Dificultad
F2Pool	BTC, ALPH, KAS, LTC...	Token V2 (POST)	command_line	Difícil
K1Pool	KAS, RXD, ZIL...	Wallet (GET)	command_line	Fácil
2Miners	ETC, RVN, KAS, ZEC...	Wallet (GET)	rest:	Fácil
ViaBTC	BTC, ETH, LTC...	API Key (header)	rest:	Medio
NiceHash	BTC (todos algos)	HMAC complejo	command_line	Difícil
HeroMiners	KAS, ALPH, RXD...	Wallet (GET)	rest:	Fácil
Kryptex	BTC, KAS, RXD...	Token (GET)	rest:	Medio

Metodo universal (cualquier nuevo pool)

1. Probar la API desde PowerShell (Invoke-RestMethod) y anotar la estructura JSON
2. Crear un sensor HA: rest: si GET simple, command_line: si POST o auth compleja
3. Crear sensores plantilla `f2pool_SIMBOLO_day` y `f2pool_SIMBOLO_unpaid`
4. Anadir la cripto a `MINING_COINS` en `secrets.js`
5. Reiniciar HA y verificar en Herramientas dev -> Estados

Ejemplo 2Miners (GET publico, wallet)

API publica sin autenticacion: la direccion wallet basta. Subdominio por cripto (etc.2miners.com, kas.2miners.com...). Devuelve balance, recompensa 24h y workers.

```
rest:
- resource: "https://etc.2miners.com/api/accounts/YOUR_WALLET"
  scan_interval: 300
  sensor:
    - name: "2miners_etc_raw"
      value_template: >
        {{ (value_json.stats.balance|float(0) / 1e9) | round(6) }}
      json_attributes: [stats, "24hreward", workersOnline]

template:
- sensor:
    - name: "f2pool_etc_day"      # generic dashboard name
      state: >
        {% set r = state_attr('sensor.2miners_etc_raw', '24hreward') %}
        {{ (r|float(0) / 1e9) | round(6) if r else 0 }}
```

Divisor segun la moneda: ETC = / 1.000.000.000, KAS = / 100.000.000, ZEC = / 100.000.000. Verifica siempre la respuesta bruta antes de elegir el divisor.

Ejemplo ViaBTC (GET + API Key header)

Clave API en el header (sin firma HMAC). Endpoints separados para hashrate y beneficios.

```
rest:
- resource: "https://www.viabtc.com/res/openapi/v1/profit?coin=BTC"
  scan_interval: 300
  headers:
    Authorization: "YOUR_API_KEY_VIABTC"
  sensor:
    - name: "viabtc_btc_profit"
      value_template: "{{ value_json.data.total_profit|float(0) }}"
```

Ejemplo NiceHash (HMAC - script Python)

NiceHash usa autentificación HMAC compleja: cada petición se firma con timestamp + nonce. Imposible directamente con sensores HA -> se usa un script Python en la Pi, llamado via `command_line`.

```
# /config/scripts/nicehash_balance.py
import hashlib, hmac, uuid, time, requests
API_KEY='...'; API_SECRET='...'; ORG_ID='...'
def sign(method, path, ts, nonce, body=''):
    msg = f'{API_KEY}\x00{ts}\x00{nonce}\x00\x00{ORG_ID}'
    msg += f'\x00\x00{method}\x00{path}\x00{body}'
    return hmac.new(API_SECRET.encode(), msg.encode(),
                    hashlib.sha256).hexdigest()
ts=str(int(time.time()*1000)); nonce=str(uuid.uuid4())
path='/main/api/v2/accounting/account2/BTC'
sig=sign('GET', path, ts, nonce)
headers={'X-Time':ts, 'X-Nonce':nonce, 'X-Organization-Id':ORG_ID,
        'X-Auth':f'{API_KEY}:{sig}'}
r=requests.get(f'https://api2.nicehash.com{path}', headers=headers)
print(r.json().get('totalBalance',{}).get('available','0'))
```

```
# configuration.yaml
command_line:
  - sensor:
      name: "nicehash_btc_balance"
      scan_interval: 300
      command: "python3 /config/scripts/nicehash_balance.py"
      value_template: "{{ value | float(0) }}"
      command_timeout: 20
```

Endpoints de los otros pools

Pool	Endpoint	Campos clave
K1Pool	GET /api/miner/{coin}/{wallet}	miner.pendingBalance, miner.coinsPerDay
HeroMiners	GET /api/stats_address?address={w}	stats.balance, stats.paid_today
Kryptex	GET /api/v1/hashrates?token={t}	data.balance, data.todayIncome
F2Pool	POST /v2/assets/balance	balance, yesterday_income

10 Acceso remoto

Tailscale VPN

Tailscale crea una red privada entre tus dispositivos. Accedes a tu dashboard desde cualquier lugar (4G, WiFi externo) como si estuvieras en casa, sin abrir ningún puerto.

#	Paso	Detalle
1	Add-on HA	Ajustes -> Add-ons -> Tailscale -> Iniciar
2	Cuenta Tailscale	tailscale.com (gratis) -> anotar la IP 100.x.x.x
3	App movil	Instalar Tailscale iOS/Android, misma cuenta
4	Editar secrets.js	Poner HA_URL_VPN con tu IP 100.x.x.x

```
const HA_URL_LOCAL = 'http://192.168.1.XX:8123';
const HA_URL_VPN   = 'http://100.X.X.X:8123';
// Auto-switch based on detected network
let HA_URL = location.hostname.startsWith('192.168')
  ? HA_URL_LOCAL : HA_URL_VPN;
```

Cambio automatico: el dashboard detecta si se abre localmente (192.168.x.x) o en remoto, y elige la URL adecuada. Transparente para el usuario.

11

Resolucion de problemas

FAQ y solucion de problemas

Problema	Solucion
El dashboard muestra ERROR	Token invalido o IP incorrecta. Recrea el token en HA y verifica la URL en secrets.js.
Sensores unavailable	Recarga las plantillas: Herramientas dev -> YAML -> Entidades de plantilla.
Los ASIC no se encienden	Verifica que la automatizacion V37 este activa y que el excedente supere el umbral durante la duracion requerida.
IDs de enchufes cambiados	Herramientas dev -> Estados -> filtrar 'switch.' para encontrar los nuevos entity_id.
Grafico dia vacio	Normal las primeras 24h. El historial se acumula progresivamente.
Grafico semana/mes vacio	Verifica recorder: purge_keep_days >= 31 y que los sensores esten en include:.
Importacion de red pese al excedente	El filtro ignora importacion < 20W (ruido CT Refoss). Mas alla, verifica la calibracion de las pinzas.
Precios cripto unknown	Espera 10 min (1a llamada CoinGecko) y verifica el acceso a internet de la Pi.
Informe 20h no recibido	Verifica las credenciales Telegram/ntfy y que la automatizacion del informe este activa. Pruebala manualmente.
Yo-yo de ASIC	Aumenta el cooldown (30s) o el retardo de arranque prioritario (10 min) en la automatizacion.
Inaccesible en 4G	Tailscale debe estar activo en el telefono Y en HA. Verifica la IP 100.x.x.x en la app.

12 Buena instalacion!

Este proyecto se comparte gratis. Mejoralo, distribuyelo, compartelo con la comunidad.

Lista final

- [V] Refoss EM06P integrado en HA (6 sensores de potencia visibles)
- [V] configuration.yaml con plantillas, ASIC prioritarios e ingresos
- [V] Automatizacion V1.0.37 activa (multi-prioridad, SHED, anti yo-yo)
- [V] Automatizacion informe diario 20h configurada (Telegram/ntfy/HA)
- [V] Dashboard HTML personalizado en /config/www/
- [V] secrets.js relleno (nunca subido a GitHub)
- [V] Acceso remoto Tailscale (opcional)

Solar ASIC v4 - Halo44 - Licencia libre.

Minemos limpio. Código fuente y ejemplos disponibles en GitHub.